

✅ 📅 DAY 8 — MULTI-TOOL AGENT (SMART ROUTING)

🎯 GOAL

Right now your system does:

if "excel" → Excel

else → RAG

👉 That is rule-based ❌

✅ After Day 8:

Query

↓

Agent decides automatically ✅ 🔥

├── Excel tool ✅

└── RAG ✅

👉 No manual keywords needed ✅

💡 CORE IDEA

You will build a decision agent that asks:

"Which tool should I use?"

✅ STEP 1 — CREATE AGENT FILE

📁 app/agent.py

✅ FULL CODE (with explanation)

```
from models.llm import generate
```

```
class Agent:
```

```
    def init(self, rag_system, excel_agent=None):
```

```
self.rag = rag_system
self.excel = excel_agent
```

```
# -----
```

```
#  TOOL DECISION
```

```
# -----
```

```
def choose_tool(self, query):
```

```
    """
```

```
    Ask LLM to decide which tool to use
```

```
    """
```

```
    prompt = f"""
```

You are an intelligent router.

Choose the best tool for the query.

Available tools:

1. RAG → for general knowledge / documents
2. EXCEL → for data analysis (numbers, averages, tables)

Rules:

- If query involves numbers, averages, totals → EXCEL
- Otherwise → RAG

Query:

{query}

Return ONLY one word: RAG or EXCEL

"""

```
decision = generate(prompt).strip().upper()
```

```
return decision
```

```
# -----
```

```
#  MAIN EXECUTION
```

```
# -----
```

```
def run(self, query):
```

```
    tool = self.choose_tool(query)
```

```
    print(f"[Agent] Selected tool: {tool}")
```

```
    #  Use Excel Agent
```

```
    if tool == "EXCEL" and self.excel:
```

```
        return self.excel.smart_query(query, generate)
```

```
    #  Default → RAG
```

```
    results = self.rag.search(query)
```

```
    context = "\n".join(results[:3])
```

```
    response = generate(f"""
```

Answer using context:

{context}

Question: {query}

""")

return response

🧠 UNDERSTAND THIS

- ◆ choose_tool()

It does:

Query → LLM → decide tool

- ◆ run()

It executes:

Decision → call correct system ✅

✅ ✅ STEP 2 — UPDATE MAIN

📁 app/main.py

✅ IMPORT AGENT

from app.agent import Agent

✅ INITIALIZE AGENT

Replace your Excel + RAG usage with:

agent = Agent(vs, excel_agent)

✅ REPLACE LOOP LOGIC

❌ OLD:

if "excel" in query:

...

✅ NEW:

while True:

 query = input(">> ")

 if query.lower() == "exit":

 break

 response = agent.run(query)

 print("\nAnswer:\n", response)

✅ ✅ STEP 3 — TEST

Run:

python -m app.main

✅ Try queries:

>> what is artificial intelligence

👉 should go to RAG

>> average sales

👉 should go to EXCEL

>> summarize document

👉 RAG

>> max profit

👉 EXCEL

✅ EXPECTED OUTPUT

[Agent] Selected tool: EXCEL

or

[Agent] Selected tool: RAG

🧠 WHAT YOU JUST BUILT

👉 This is VERY IMPORTANT:

You built a decision-making AI system ✅

🔥 BEFORE vs AFTER

✗ Before:

if "excel" in query → tool

✅ Now:

LLM decides tool ✅ 🔥

🧠 REAL INDUSTRY CONCEPT

This is called:

✅ Tool Routing Agent / Router Agent

Used in:

- ChatGPT plugins
 - Copilot
 - enterprise assistants
-

✅✅ OPTIONAL IMPROVEMENT (SMALL BUT POWERFUL)

Add fallback:

if tool not in ["RAG", "EXCEL"]:

tool = "RAG"

✅✅ FINAL SYSTEM NOW

User Query

↓

Agent (LLM)

↓

Decide tool

|—— Excel Agent ✅

|—— RAG System ✅

🔥 YOUR SYSTEM LEVEL NOW

You built:

- ✅ Hybrid RAG
 - ✅ Reranker
 - ✅ Excel Agent
 - ✅ Smart Agent Routing ✅ 🔥
-

🚀 NEXT (DAY 9)

We will go to:

🔥 MULTI-STEP AGENT (VERY POWERFUL)

Example:

"Compare pdf revenue with excel data"

👉 agent will call both tools ✅ 🔥

💬 Say:

👉 Day 9

100 BIG milestone —

You are now building real AI agents, not just pipelines 🚀

I modified the agent code using claude to make it run

```
import pandas as pd
```

```
import numpy as np
```

```
import re
```

```
class ExcelAgent:
```

```
    def __init__(self, df):
```

```
        self.raw_df = df
```

```
        self.df = self._clean_data(df)
```

```
        self.schema = self._analyze_schema()
```

```
    def _clean_data(self, df):
```

```
        df = df.copy()
```

```
        df.columns = df.columns.str.strip().str.lower()
```

```
        for col in df.columns:
```

```
            if pd.api.types.is_numeric_dtype(df[col]):
```

```
                df[col] = df[col].fillna(df[col].median())
```



```
else:
```

```
    df[col]=df[col].fillna("unknown")
```

```
# for col in df.columns:
```

```
#     df[col] = pd.to_numeric(df[col], errors = "coerce")
```

```
return df
```

```
def _analyze_schema(self):
```

```
    schema = {}
```

```
    for col in self.df.columns:
```

```
        if pd.api.types.is_numeric_dtype(self.df[col]):
```

```
            schema[col] = "numeric"
```

```
        elif self.df[col].nunique()<20:
```

```
            schema[col] = "categorical"
```

```
        else:
```

```
            schema[col]= "text"
```

```
    return schema
```

```
def summary(self):
```

```
    report = []
```

```
    report.append("📊 DATASET OVERVIEW")
```

```
    report.append(f"Rows: {len(self.df)}, Columns: {len(self.df.columns)}\n")
```

```
    for col, typ in self.schema.items():
```

```
        if typ == "numeric":
```

```

        report.append(f"📊 {col} (Numeric) - Mean: {self.df[col].mean():.2f}\n,Max={self.df[col].max():.2f}\n,Min={self.df[col].min():.2f}\n,Median: {self.df[col].median():.2f}\n, Std: {self.df[col].std():.2f}")

```

```

for col, typ in self.schema.items():

```

```

    if typ == "categorical":

```

```

        report.append(f"📁 {col} (Categorical) - Unique Values: {self.df[col].nunique()}\n,Top 5: {self.df[col].value_counts().head().to_dict()}")

```

```

return "\n".join(report)

```

```

def _clean_code(self, code):

```

```

    # remove markdown blocks

```

```

    code = re.sub(r"```python|```", "", code)

```

```

    # remove lines that are comments or explanations

```

```

    lines = code.splitlines()

```

```

    code_lines = []

```

```

    for line in lines:

```

```

        line = line.strip()

```

```

        if not line:

```

```

            continue

```

```

        if line.startswith("#"):

```

```

            continue

```

```

        if line.startswith("Here") or line.startswith("This") or line.startswith("The"):

```

```

            continue

```

```

        code_lines.append(line)

```

```

    # take the last line — it's almost always the expression

```

```

    return code_lines[-1] if code_lines else ""

```

```

def smart_query(self, question, llm_func):

```

```
schema_desc = "\n".join([f"{k}: {v}" for k, v in self.schema.items()])
```

```
last_error = None
```

```
for attempt in range(3):
```

```
    error_hint = f"\nPrevious attempt failed with error: {last_error}" if last_error else ""
```

```
    prompt = f"""
```

You are a pandas code generator.

DataFrame name: df

Schema: {schema_desc}

IMPORTANT:

- Return ONLY valid Python pandas code
- No explanation, No imports, No markdown
- Must be a valid python expression

Question: {question}

{error_hint}

```
"""
```

```
    raw_code = llm_func(prompt)
```

```
    code = self._clean_code(raw_code)
```

```
    if not code:
```

```
        last_error = "Empty code returned"
```

```
        continue
```

```
try:
```

```
    #  safe eval (restricted scope)
```

```
    result = eval(
```

```

        code,

        {"__builtins__": {}},

        {"df": self.df, "pd": pd, "np": np}

    )

    explanation = llm_func(f"""

```

Result of pandas query: {str(result)[:500]}

Question asked: {question}

Give a clear 1-2 sentence answer. No code.

```

""")

```

```

        return explanation

```

```

    except Exception as e:

```

```

        last_error = str(e)

```

```

        continue

```

```

    return f"Execution failed after 3 attempts: {last_error or 'unknown error'}"

```

```

# -----

```

```

#  QUERY SUGGESTIONS

```

```

# -----

```

```

def suggest_questions(self):

```

```

    suggestions = []

```

```

    numeric_cols = [c for c, t in self.schema.items() if t == "numeric"]

```

```

    cat_cols = [c for c, t in self.schema.items() if t == "categorical"]

```

```

    if numeric_cols:

```

```

        suggestions.append(f"average of {numeric_cols[0]}")

```

```

        suggestions.append(f"top values in {numeric_cols[0]}")

```

```
if len(numeric_cols) >= 2:
    suggestions.append(
        f"correlation between {numeric_cols[0]} and {numeric_cols[1]}"
    )

if cat_cols and numeric_cols:
    suggestions.append(
        f"average {numeric_cols[0]} by {cat_cols[0]}"
    )

return suggestions
```